

A1. Анализ строковых сортировок

A1. Анализ строковых сортировок

Учебные подразделения
Алгоритмы и структуры данных (2025/2026 модули: 1,2,3,4) (Нестеров Р.А.)
A1. Анализ строковых сортировок

Открыто с: понедельник, 11 мая 2026, 19:00
Срок сдачи: понедельник, 25 мая 2026, 02:00

Стандартная сложность алгоритмов сортировки, которые работают на основе сравнений, составляет $\Omega(n \cdot \log n)$. Однако при сортировке строк эта оценка требует корректировки, поскольку приходится выполнять сравнение строк посимвольно. Для эффективной сортировки строк используются специальные методы, основанные на работе с префиксами строк. Алгоритмы учитывают совпадающие и различающиеся начальные части строк, что позволяет избежать лишних посимвольных сравнений при сортировке.

Вам необходимо провести практическое исследование, в ходе которого нужно сравнить работу обычных и модифицированных алгоритмов сортировки для массивов строк. Сортировка должна выполняться в прямом лексикографическом порядке.

- Реализация алгоритмов должна быть выполнена на языке программирования C++.
- Для обработки и визуализации полученных эмпирических результатов можно использовать любые доступные инструменты.

Этап 1. Подготовка тестовых данных

Разработайте класс `StringGenerator` для генерации массивов случайных строк. При генерации будут использоваться следующие символы (всего 74 знака):

- заглавные и строчные латинские буквы `A..Z, a..z`;
- цифры `0123456789`;
- дополнительные специальные символы `!@#%:;^&*()_-`.

Параметры генерации определяются следующим образом:

РЕШЕНИЕ

Что делаем

В задаче нужно сравнить несколько алгоритмов сортировки строк

Обычная сортировка считает, что сравнение двух элементов стоит $O(1)$
Для строк это не так

Чтобы сравнить две строки, надо идти по символам слева направо, пока не найдется отличие или пока одна строка не закончится

Поэтому на строках с длинными общими префиксами обычные сортировки могут работать заметно хуже

Сравниваем сравниваются:

QUICKSORT
MERGESORT
STRING QUICK SORT
STRING MERGE SORT
MSD RADIX SORT
MSD RADIX SORT + QUICK SORT

Смотрим два параметра:

время работы
количество посимвольных сравнений

Для генерации тестов был написан StringGenerator

Алфавит:

A..Z
a..z
0..9
!@#%: ^&*()-_

В условии написано 74 символа

Перечисленные символы дают 73, поэтому я добавил _, чтобы получилось 74

Длина строки: от 10 до 200 символов

Размер массива: от 100 до 3000 строк шаг 100

Типы массивов:

случайный массив
обратно отсортированный массив
почти отсортированный массив
массив с общим префиксом

Случайный массив

Все строки генерируются независимо и случайно

Обратно отсортированный массив

Сначала генерируем случайный массив
Потом сортируем его по возрастанию
Потом разворачиваем

Почти отсортированный массив

Сначала сортируем массив
Потом делаем небольшое количество случайных swap

Массив с общим префиксом

У всех строк есть одинаковое начало
Этот тест важен, потому что обычное сравнение строк на нем долго идет по одинаковым символам

QUICKSORT

Что делаем:

берем опорный элемент
идем двумя указателями
слева ищем элемент, который надо перенести вправо
справа ищем элемент, который надо перенести влево
меняем элементы местами
рекурсивно сортируем левую и правую части

Опорный элемент:

середина текущего отрезка

Сравнение строк:

лексикографическое
с подсчетом сравненных символов

Сложность: в среднем $O(n \log n)$ сравнений в худшем случае $O(n^2)$

Но для строк надо учитывать длину сравнения
Если у строк длинный общий префикс, одно сравнение может стоить $O(L)$

MERGESORT

Базовенькая сортировка слиянием

Что делаем:

делим массив на две части
сортируем левую часть
сортируем правую часть
сливаем две отсортированные части. При слиянии строки сравниваются лексикографически

Сложность:

$O(n \log n)$ сравнений

Плюс надо учитывать стоимость сравнения строк

Плюсы:

стабильная асимптотика
нет плохого случая как у quicksort

Минусы:

нужна дополнительная память
одинаковые префиксы могут сравниваться много раз

STRING QUICK SORT

Это тернарная быстрая сортировка строк

Главная идея:

сравниваем строки не целиком, а по символу на позиции d

На текущем шаге строки делятся на три группы:

символ меньше опорного
символ равен опорному
символ больше опорного

Дальше: левая часть сортируется по той же позиции d
средняя часть сортируется по позиции $d + 1$
правая часть сортируется по той же позиции d

Почему это хорошо

Если у строк общий префикс, алгоритм не сравнивает его заново много раз
Он просто переходит к следующей позиции внутри средней группы

На случайных строках разница может быть не очень большой
На строках с общим префиксом разница получилась заметной: STRING QUICK SORT дал намного меньше сравнений и лучшее время на prefix-тесте

STRING MERGE SORT

Это сортировка слиянием, где используется LCP

LCP - длина наибольшего общего префикса

Пример. Abcde. abcfg

LCP = 3

Потому что общий префикс:

abc

Главная идея:

если известно, что первые несколько символов совпадают, то не надо сравнивать их заново

Можно начинать сравнение с первой неизвестной позиции

На строках с общим префиксом это должно уменьшать число повторных сравнений, но в моем эксперименте по времени STRING MERGE SORT не всегда оказался быстрее обычного MERGESORT из-за дополнительных вычислений LCP

На случайных строках эффект меньше, потому что различие часто находится почти сразу

MSD RADIX SORT

MSD RADIX SORT не сравнивает строки друг с другом

Он распределяет строки по символам

MSD значит, что начинаем с самого левого символа

Что делаем:

раскладываем строки по первому символу
каждую группу сортируем по второму символу
потом по третьему
и так далее

Для конца строки используется отдельное значение меньше всех символов

Это нужно для правильного порядка

Пример. abc
abcd

Правильный порядок. Abc. abcd

Потому что короткая строка должна идти раньше своей длинной версии

Количество посимвольных сравнений для MSD RADIX SORT равно 0

Но это не значит, что он ничего не делает

Он обращается к символам строк, просто не сравнивает строки между собой

MSD RADIX SORT + QUICK SORT

Это смешанный алгоритм

Что делаем:

на больших подмассивах используем MSD RADIX SORT
на маленьких подмассивах переходим на quicksort

Граница переключения: 74

Почему 74:

это размер алфавита

Зачем это нужно:

на маленьких подмассивах radix sort может тратить много лишних действий на корзины

Quicksort на маленьком массиве часто быстрее

Поэтому смешанный алгоритм убирает часть накладных расходов. Измерения:

Для измерений был написан StringSortTester

Он делает так:

берет исходный массив
копирует его для каждого алгоритма
запускает сортировку
измеряет время
считает посимвольные сравнения
печатает результат

Формат вывода:

type;n;algorithm;time_microseconds;char_comparisons

Наприме

random;100;QUICKSORT;120;4500

Обычные quicksort и mergesort на случайных данных работают нормально, но по графикам видно, что QUICKSORT все равно делает больше всего посимвольных сравнений

Строки часто отличаются на первых символах, поэтому разница между алгоритмами меньше, чем на prefix-тесте

Обратно отсортированные массивы

Mergesort работает стабильно

Quicksort на обратно отсортированных данных оказался самым дорогим по времени и числу сравнений, хотя опорный элемент берется из середины

Почти отсортированные массивы

Большого выигрыша обычно нет

Массив почти отсортирован, но реализованные алгоритмы не используют это напрямую

Массивы с общим префиксом

Тут обычным сортировкам тяжелее

Причина. каждое сравнение строк долго проходит по одинаковому началу

Лучше по числу сравнений должны работать

STRING QUICK SORT

STRING MERGE SORT

MSD RADIX SORT

MSD RADIX SORT + QUICK SORT

По времени на prefix-тесте лучше всего показал себя STRING QUICK SORT, а на остальных типах данных часто быстрее был MSD RADIX SORT + QUICK SORT

Итоговое сравнение

QUICKSORT

Плюсы:

простая реализация
обычно быстро работает
не нужен дополнительный массив размера n

Минусы:

есть худший случай $O(n^2)$
может много раз сравнивать одинаковые префиксы

MERGESORT

Плюсы:

всегда $O(n \log n)$ сравнений
стабильно работает на разных входах

Минусы:

нужна дополнительная память
общие префиксы могут проверяться много раз

STRING QUICK SORT

Плюсы:

хорошо подходит для строк
не гоняет общий префикс заново много раз
хорош на строках с одинаковым началом

Минусы:

сложнее обычного quicksort
зависит от распределения символов

STRING MERGE SORT

Плюсы:

использует LCP
меньше повторных сравнений
сохраняет идею mergesort

Минусы:

реализация сложнее
нужно аккуратно считать LCP

MSD RADIX SORT

Плюсы:

не сравнивает строки напрямую
хорошо подходит для строк
эффективен при общих префиксах

Минусы:

нужна дополнительная память
есть расходы на корзины
на маленьких подмассивах может быть невыгоден

MSD RADIX SORT + QUICK SORT

Плюсы:

уменьшает расходы radix sort
хорошо работает на больших группах
на маленьких группах использует более простой алгоритм

Минусы:

надо выбрать границу переключения
реализация сложнее обычной сортировки

ВЫВОД

Обычные QUICKSORT и MERGESORT хорошо подходят как базовые алгоритмы
На случайных строках они работают нормально, потому что строки часто различаются в начале

Но на строках с длинными общими префиксами обычные сортировки делают много лишней работы
Они снова и снова сравнивают одинаковые символы в начале строк

STRING QUICK SORT решает это тем, что сортирует строки по символам и переходит к следующей позиции только внутри группы равных символов

STRING MERGE SORT использует LCP и не сравнивает заново уже известный общий префикс

MSD RADIX SORT вообще не сравнивает строки напрямую
Он распределяет их по символам, поэтому число посимвольных сравнений строк у него равно 0

MSD RADIX SORT + QUICK SORT нужен, чтобы не тратить лишнее время на radix sort в маленьких подмассивах. На моих графиках он чаще всего оказался самым быстрым алгоритмом по времени

ОТВЕТ

Сделай:

StringGenerator

StringSortTester

QUICKSORT

MERGESORT

STRING QUICK SORT

STRING MERGE SORT

MSD RADIX SORT

MSD RADIX SORT + QUICK SORT

Главный вывод:

на обычных случайных строках стандартные сортировки работают нормально, но QUICKSORT делает больше всего посимвольных сравнений

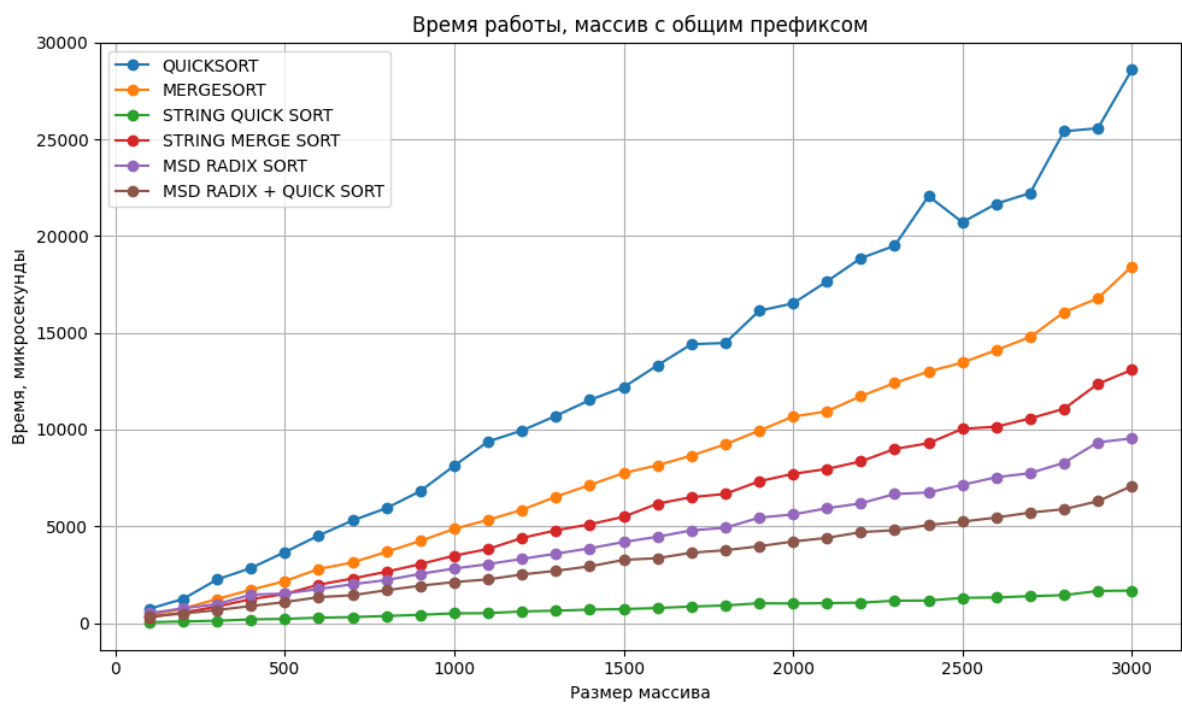
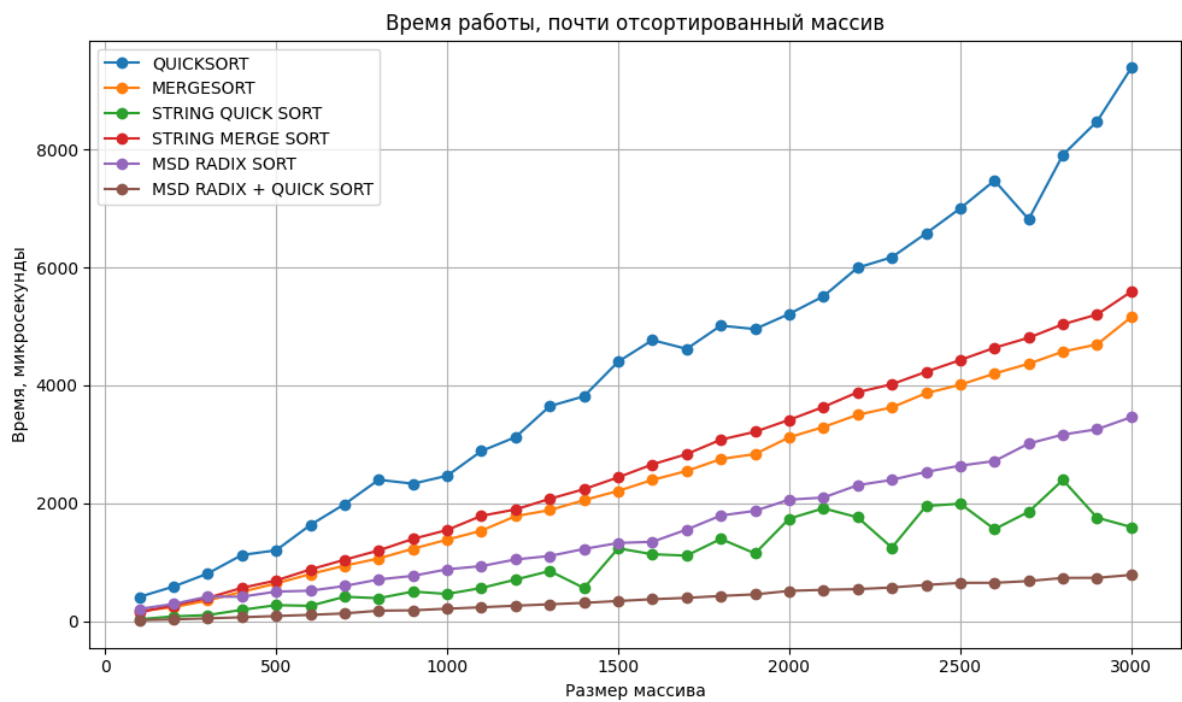
На строках с общими префиксами лучше подходят специальные строковые сортировки, потому что они меньше раз проходят по одним и тем же символам

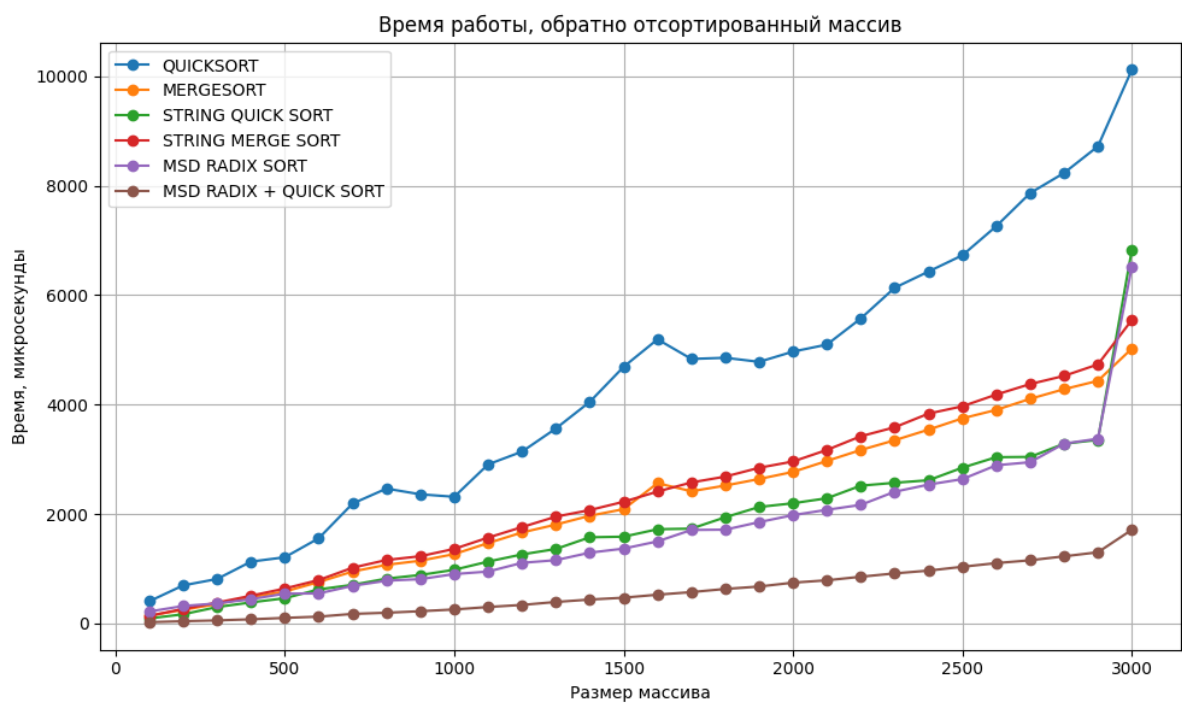
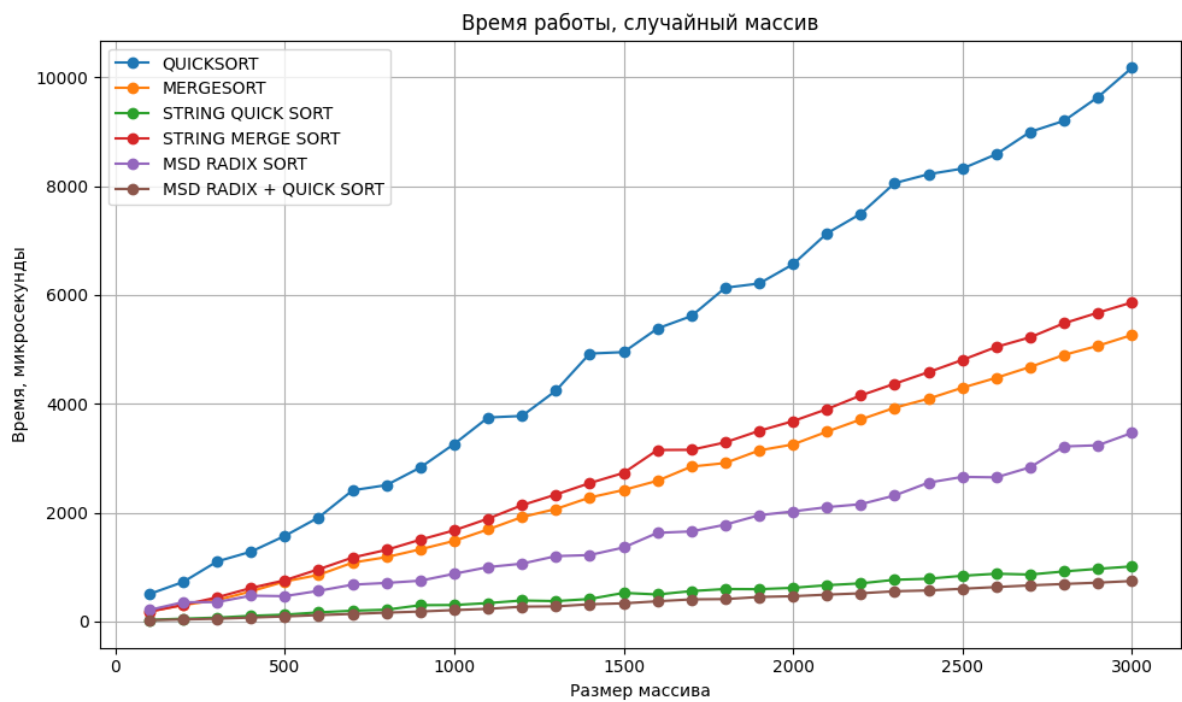
По времени чаще всего лучшим оказался MSD RADIX SORT + QUICK SORT, а на prefix-тесте лучше всего показал себя STRING QUICK SORT

Код

		time_microseconds	char_comparisons
type	algorithm		
almost	MERGESORT	2392.633333	18120.666667
	MSD_RADIX_QUICK_SORT	375.466667	7246.666667
	MSD_RADIX_SORT	1574.133333	0.000000
	QUICKSORT	4290.866667	211763.866667
	STRING_MERGE_SORT	2642.766667	31098.300000
	STRING_QUICK_SORT	1042.733333	40961.500000
prefix	MERGESORT	8199.200000	313178.900000
	MSD_RADIX_QUICK_SORT	3308.100000	5586.700000
	MSD_RADIX_SORT	4498.500000	0.000000
	QUICKSORT	13077.466667	649987.933333
	STRING_MERGE_SORT	5926.133333	327039.200000
	STRING_QUICK_SORT	782.900000	46553.400000
random	MERGESORT	2571.033333	20791.500000
	MSD_RADIX_QUICK_SORT	361.733333	6079.266667
	MSD_RADIX_SORT	1584.433333	0.000000
	QUICKSORT	5182.033333	244074.866667
	STRING_MERGE_SORT	2882.000000	35676.133333
	STRING_QUICK_SORT	496.966667	16461.100000
reverse	MERGESORT	2247.766667	13215.400000

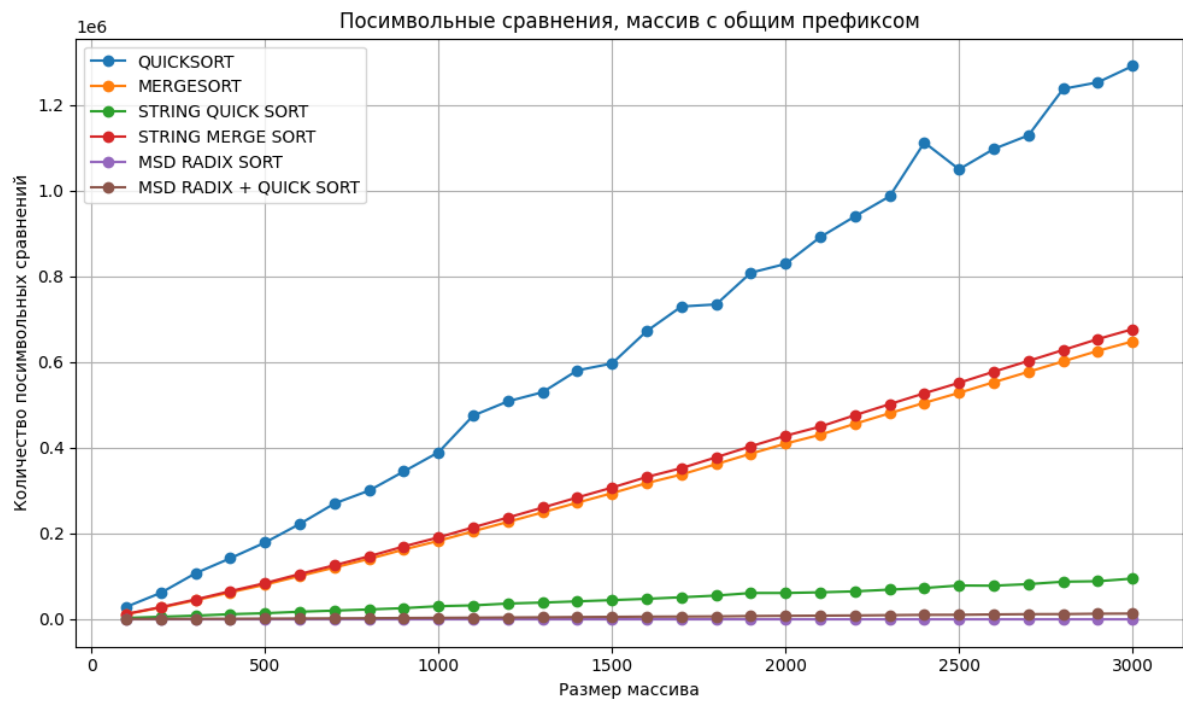
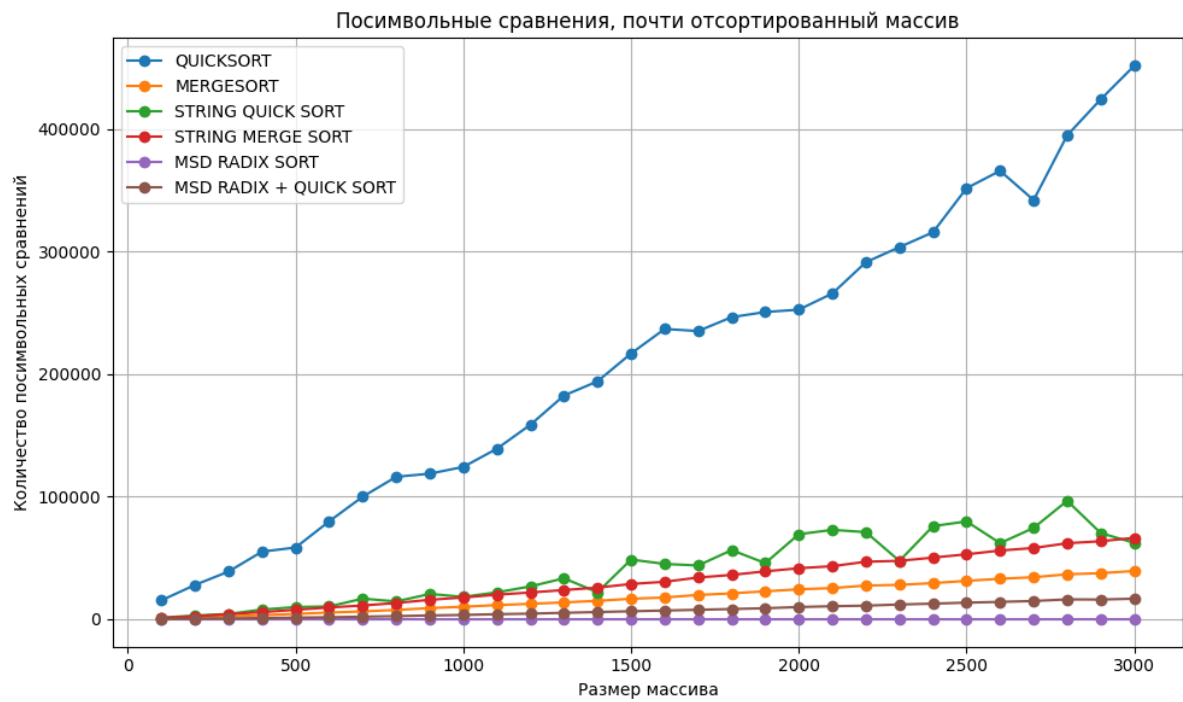
Графики

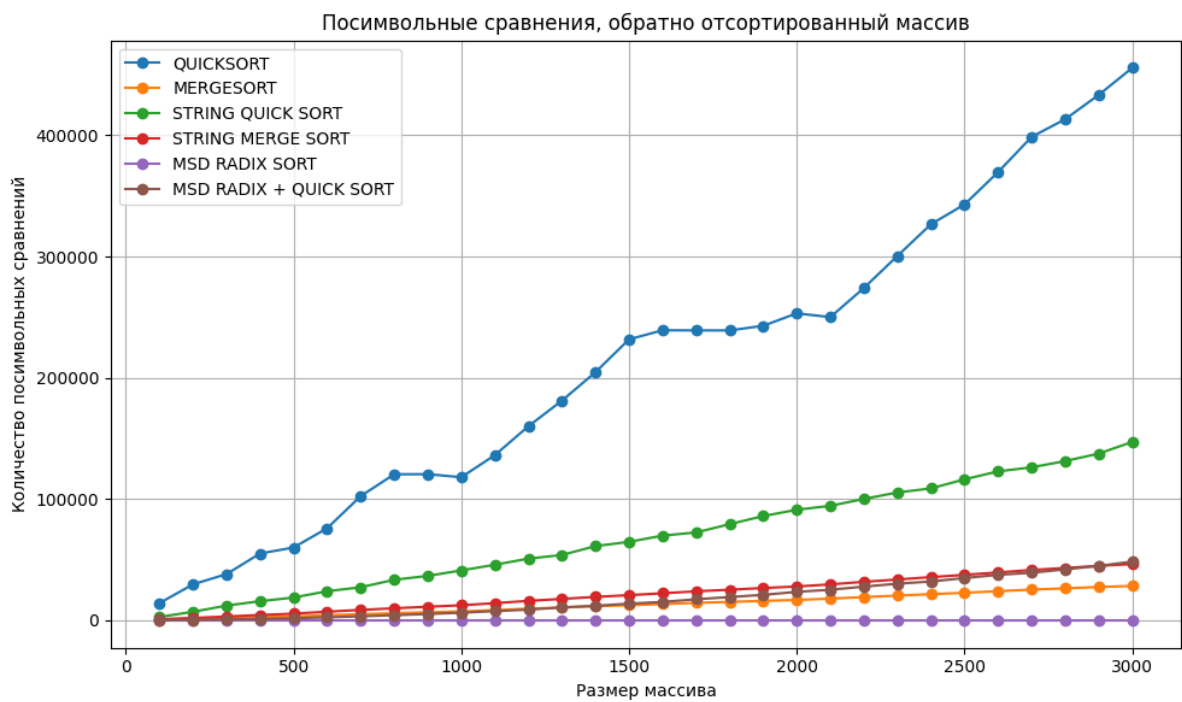
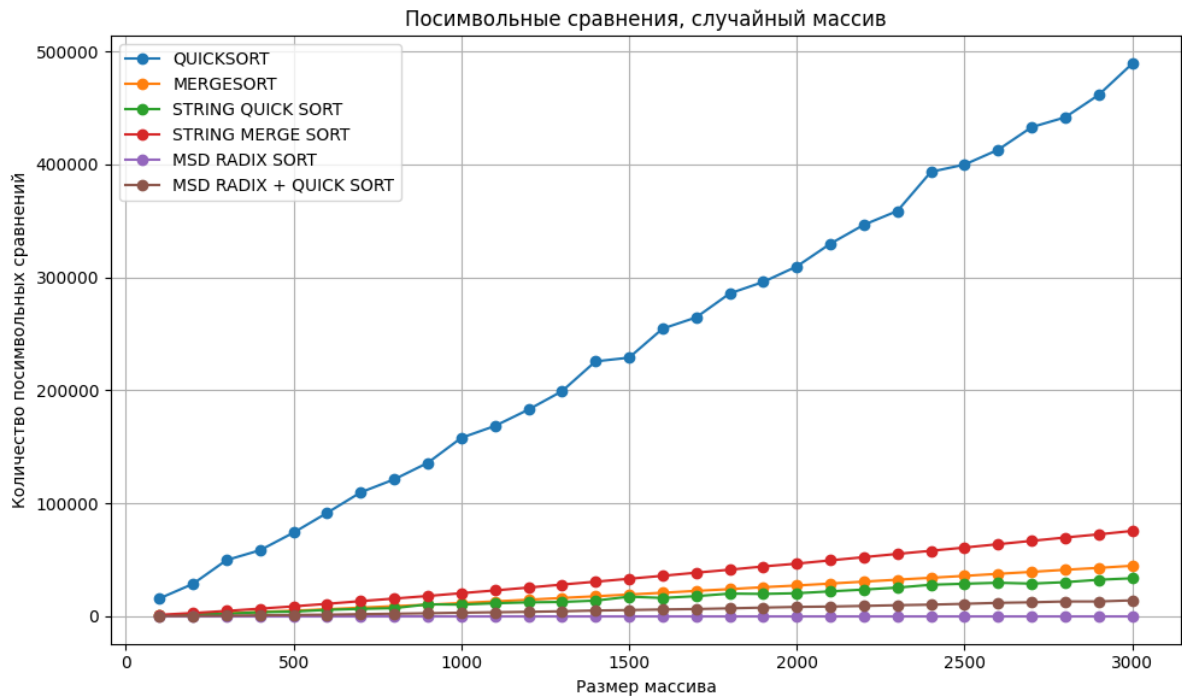




И еще

пуруру





Мои посылки					
№	Когда	Кто	Задача	Язык	Вердикт
375977903	13 дней	Капогузов Максим Евгеньевич	A1rq - Реализация MSB RADIX+QUICK SORT	C++20 (GCC 13-64)	Полное решение: 3 баллов
375977893	13 дней	Капогузов Максим Евгеньевич	A1rq - Реализация MSB RADIX+QUICK SORT	C++20 (GCC 13-64)	Ошибка компиляции
375977876	13 дней	Капогузов Максим Евгеньевич	A1r - Реализация MSB RADIX SORT	C++20 (GCC 13-64)	Полное решение: 4 баллов
375977866	13 дней	Капогузов Максим Евгеньевич	A1q - Реализация STRING QUICK SORT	C++20 (GCC 13-64)	Полное решение: 4 баллов
375977857	13 дней	Капогузов Максим Евгеньевич	A1m - Реализация STRING MERGE SORT	C++20 (GCC 13-64)	Полное решение: 5 баллов

